

## Zad.1 Część 1

```

--- WYŚCIG: 2 wątki/ów ---
Oczekiwana: 2,000,000
Rzeczywista: 1,620,971
Utracone: 379,029 (18.95%)
Czas: 0.06 s

```

Naciśnij Enter...

```

--- WYŚCIG: 4 wątki/ów ---
Oczekiwana: 4,000,000
Rzeczywista: 2,247,097
Utracone: 1,752,903 (43.82%)
Czas: 0.13 s

```

Naciśnij Enter...

```

--- WYŚCIG: 8 wątki/ów ---
Oczekiwana: 8,000,000
Rzeczywista: 3,885,420
Utracone: 4,114,580 (51.43%)
Czas: 0.22 s

```

Naciśnij Enter...

```

--- WYŚCIG: 16 wątki/ów ---
Oczekiwana: 16,000,000
Rzeczywista: 10,662,215
Utracone: 5,337,785 (33.36%)
Czas: 0.41 s

```

| Liczba Wątków | Utrata wartości counter (%) |
|---------------|-----------------------------|
| 2             | 18.95%                      |
| 4             | 43.82%                      |
| 8             | 51.43%                      |
| 16            | 33.36%                      |

b. Im więcej wątków tym więcej razy system operacyjny wywoła tą funkcję często w nieodpowiednim momencie co z powodu nieprzystosowania pythona do tego typu działań co może skutkować odczytem starej wartości przez wątek, zawieszeniem jego działania, w międzyczasie inne wątki zmieniają wartość zmiennej a na koniec początkowy wątek nadpisuje ich prace swoją wartością.

c. Po zmianie `get_one()` na prostą inkrementację zmiennej `counter` utrata wartości zmiennej jest równa zero we wszystkich przypadkach (zdjęcie poniżej). W tym przypadku wątek nie musi być zawieszany więc praca innych wątków nie psuje wyniku. Wyścig powinien zajść ale przez implementację Pythona został on przed nami ukryty przez twórców.

```
--- WYŚCIG: 2 wątki/ów ---  
Oczekiwana: 2,000,000  
Rzeczywista: 2,000,000  
Utracone: 0 (0.00%)  
Czas: 0.04 s  
  
Naciśnij Enter...  
  
--- WYŚCIG: 4 wątki/ów ---  
Oczekiwana: 4,000,000  
Rzeczywista: 4,000,000  
Utracone: 0 (0.00%)  
Czas: 0.09 s  
  
Naciśnij Enter...  
  
--- WYŚCIG: 8 wątki/ów ---  
Oczekiwana: 8,000,000  
Rzeczywista: 8,000,000  
Utracone: 0 (0.00%)  
Czas: 0.15 s  
  
Naciśnij Enter...  
  
--- WYŚCIG: 16 wątki/ów ---  
Oczekiwana: 16,000,000  
Rzeczywista: 16,000,000  
Utracone: 0 (0.00%)  
Czas: 0.25 s  
  
Naciśnij Enter...
```

## Zad.1 Część 2

```
--- BLOKADA (poprawny): 2 wątki/ów ---
Oczekiwana: 2,000,000
Rzeczywista: 2,000,000
Błąd: 0 (powinno być 0!)
Czas: 0.14 s
```

Naciśnij Enter...

```
--- BLOKADA (poprawny): 4 wątki/ów ---
Oczekiwana: 4,000,000
Rzeczywista: 4,000,000
Błąd: 0 (powinno być 0!)
Czas: 0.31 s
```

Naciśnij Enter...

```
--- BLOKADA (poprawny): 8 wątki/ów ---
Oczekiwana: 8,000,000
Rzeczywista: 8,000,000
Błąd: 0 (powinno być 0!)
Czas: 0.58 s
```

Naciśnij Enter...

```
--- BLOKADA (poprawny): 16 wątki/ów ---
Oczekiwana: 16,000,000
Rzeczywista: 16,000,000
Błąd: 0 (powinno być 0!)
Czas: 1.14 s
```

Naciśnij Enter...

- Wyniki są poprawne, dla 8 wątków błąd tak samo jak dla wszystkich innych, jest równy zero.
- Tworzy mutex, który chroni dostęp do zmiennej counter2 aby uniknąć sytuacji z części 1 pdpkt a gdzie do zmiennej dostęp na raz miało kilka wątków co powodowało błędy.

- c. Wątki nie działają równoległe przez istnienie blokady w postaci muteksu. Czasy są około 4-5 raza większe porównując z czasami z części 1 pdpkt c i około 3 razy większe niż części pdpkt a.
- d. Różnica wacha się między 400% a 500% porównując częścią 1 pdpkt c , a porównując z pdpkt a części 1 jest to około 300%.

### Zad.1 Część 3

```
--- ZŁY WZORZEC (blokada przed pętlą): 2 wątki/ów ---  
Oczekiwana: 2,000,000  
Rzeczywista: 2,000,000  
Czas: 0.06 s  
  
Naciśnij Enter...  
  
--- ZŁY WZORZEC (blokada przed pętlą): 4 wątki/ów ---  
Oczekiwana: 4,000,000  
Rzeczywista: 4,000,000  
Czas: 0.13 s  
  
Naciśnij Enter...  
  
--- ZŁY WZORZEC (blokada przed pętlą): 8 wątki/ów ---  
Oczekiwana: 8,000,000  
Rzeczywista: 8,000,000  
Czas: 0.23 s  
  
Naciśnij Enter...  
  
--- ZŁY WZORZEC (blokada przed pętlą): 16 wątki/ów ---  
Oczekiwana: 16,000,000  
Rzeczywista: 16,000,000  
Czas: 0.42 s  
  
Naciśnij Enter...
```

- Wyniki są poprawne. Jest to jednak zła implementacja ponieważ blokada znajduje się przed pętlą. Przez to że wątki czekają aż blokada będzie zwolniona to praktycznie nie występuje tutaj wielowątkowość, bo wątki są wykonywane po kolei.
- Wynika to z oczekiwania przez wątki aż poprzedni skończy pracę.
- Oznacza to że instrukcje wykonują się jedna po drugiej, sekwencyjnie. Gdy jeden wątek działa inne są uśpione.

## Pytania Końcowe:

- To zależy. Wzorzec 2 jest lepszy architektonicznie, ale wykonuje się znacznie dłużej. Wzorzec 3 teoretycznie zwraca dobre wartości ale wykonuje się sekwencyjnie.
- Kiedy musielibyśmy zapisywać zmiany wprowadzane przez całą pętlę, bez rozdzielania jej efektów na części.

## Zad.2

|                | Wątki | Czas [s] | Przyspieszenie | Teoria |
|----------------|-------|----------|----------------|--------|
| Eksperyment 1: | 16    | 0.2874   | 1.03x          | 9.14x  |
| Eksperyment 2: | 16    | 0.2761   | 1.00x          | 2.91x  |
| Eksperyment 3: | 16    | 0.2778   | 1.03x          | 1.88x  |

Co prawda przyspieszenie praktycznie się nie zmienia, a jeżeli się to dzieje to ciężko stwierdzić tu jakąkolwiek zależność dlatego odnoszę się do teoretycznych wyników. Zapewne dzieje się to z powodu prostoty programu i szybkości komputera.

- Różnica przy zmianie z 70 % na 90% części równoległej dała około trzykrotny wzrost.
- Ponieważ ogranicza nas część sekwencyjna, która jest tu największa ze wszystkich eksperymentów.
- Wtedy iloraz P/N -> 0 więc  $S = 1 / (1 - 0.7) = 1 / 0.3 = \text{ok. } 3.33$   
Dla 16 wątków wynik teoretyczny wyniósł 2.91x więc jesteśmy dość blisko teoretycznej bariery 3.33x
- Dokupienie procesora:

$$S = 1 / (0.1 + 0.9 / 32) = \text{ok. } 7.8$$

Optymalizacja kodu:

$$S = 1 / (0.05 + 0.95 / 16) = \text{ok. } 9.14$$

Więc bardziej opłaca się zoptymalizować kod.

|    |        |       |       |
|----|--------|-------|-------|
| 16 | 0.2841 | 1.02x | 9.14x |
| 32 | 0.2848 | 1.00x | 7.80x |

### Zad.3

#### Eksperyment 1:

- a. Zmienna counter wyniosła 1, wszystkie wątki widziały tę samą zmienną.
- b. Nie, wszystkie wątki modyfikowały tę samą zmienną

#### Eksperyment 2:

- a. Counter był równy 0.
- b. Każdy proces miał swoją niezależną zmienną counter którą zwiększył, ale zmiany te nie dotyczyły procesu głównego który wypisał wynik czyli 0.
- c. Procesy mogą się komunikować np. za pomocą potoków lub kolejek.

#### Eksperyment 3:

- a. Szybsze jest tworzenie wątków, jak widać na zdjęciu poniżej, o około 12 razy.
- b. Ponieważ tworzenie procesu wymaga np. stworzenia pamięci własnej procesu co zabiera zasoby oraz czas systemu operacyjnego. Wątki działają już istniejącym procesie oraz tworzenie ich jest szybsze i mniej obciążające.

#### EKSPERYMENT 3: PORÓWNANIE CZASU (1000 uruchomień)

```
Czas utworzenia 1000 wątków: 0.051s
Czas utworzenia 1000 procesów: 0.610s
```

✓ Wątki są szybsze (~12× szybsze od procesów)